第 2 章 进程与线程管理

进程与线程管理负责组织用户程序的执行主体及其资源，向内连接地址空间、文件描述符和信号等内核对象，向外提供创建、执行、退出、等待和控制接口。

2.1 概述

2.1.1 StarryOS

StarryOS 已经建立了可运行的进程线程框架。每个用户执行流对应一个 `Thread`，同一进程中的线程共同持有 `ProcessData`。前者保存线程私有状态，后者保存地址空间、文件资源、进程级信号和父子关系等共享状态。

在功能上，StarryOS 已经提供了 `fork`、`clone`、`clone3`、`execve`、`exit` 和 `waitpid`，可以完成进程创建、程序映像替换、线程退出和僵尸进程回收。进程组、会话和 pidfd 也已有基本实现。因此，moss 面对的不是一片空白，而是一套主体结构已经成形，但是部分 Linux 语义仍较简化的实现。

2.1.2 base os

当用户程序只走“创建子进程、运行、退出、父进程回收”这条主路径时，StarryOS 已能正常工作。问题主要在接口组合变得复杂后暴露出来。

创建方面，`clone3` 已经存在，但参数结构长度、非法标志组合、栈参数和 pidfd 写回失败后的回滚不够严密。执行方面，基线只有 `execve`，缺少 `execveat`，对执行权限、`MS_NOEXEC` 和正在写入的可执行文件也缺少完整检查。

状态管理方面，原有 `waitpid` 主要处理已经退出的子进程，无法完整表达“停止”和“继续”，也没有 `waitid` 所需的目标选择和 `siginfo_t` 编码。权限方面，部分 UID/GID 与 capability 接口采用固定返回值或直接成功，进程身份没有真正参与 `kill`、程序执行和调试附加等判断。ptrace 尚未实现，pidfd 也没有接入 `waitid` 和 `/proc` 的观测接口。

这些问题说明，基线已经能够建立进程，却还没有把进程的身份、状态和生命周期完全串联起来。

2.1.3 moss

moss 保留 StarryOS 的主体结构，重点补齐创建、执行、退出、等待和控制接口之间的联系。主要增量如表所示。

内容	StarryOS 基线	moss 的增量
对象与状态	`Thread` 与 `ProcessData` 分离	补充凭据、停止/继续状态、子进程时间和 namespace 元数据
创建与执行	`clone3`、`execve` 已实现	补严 clone ABI 和失败回滚，增加 `execveat` 与执行约束
退出与等待	线程退出、僵尸进程和 `waitpid`	完善资源清理，增加 `waitid`、状态事件和 pidfd 等待
管理接口	进程组、会话、基础凭据与 pidfd	修正全局登记，实现有状态凭据和最小 ptrace 控制链路

表 2-1　进程与线程模块的主要增量

2.2 核心设计思想

StarryOS 已经搭起了进程线程管理的基本框架，moss 的工作是在这套框架中进一步理清状态归属和对象关系：哪些数据属于单个线程，哪些资源应由整个进程共享；进程之间如何组织；身份信息怎样参与权限判断；一个任务又如何从创建走到退出和回收。下面从这几个问题分别说明。

2.2.1

进程和线程并不是两套互不相干的对象。moss 延续 StarryOS 的做法，将状态分为两层：`Thread` 表示单个执行流，`ProcessData` 表示线程共同拥有的进程资源。创建普通线程时复用 `ProcessData`；创建新进程时再根据 clone 标志复制或共享其中的资源。

这种划分回答了一个最基本的问题：某项状态究竟属于线程，还是属于整个进程。线程名称、线程信号状态和退出字段放在 `Thread` 中；地址空间、程序路径、文件表作用域、凭据和子进程集合放在 `ProcessData` 中。只要边界清楚，clone、exec 和 exit 才不会各自维护一份互相冲突的状态。

2.2.2

moss 同时维护三类关系。

- 归属关系：多个线程通过共享 `ProcessData` 归属于同一进程。
- 亲缘关系：进程通过 parent 和 children 形成父子树，供退出托管和 wait 使用。
- 分组关系：进程依次归入进程组和会话，形成 PID、PGID、SID 三层标识。

这三类关系用途不同。线程组解决“哪些执行流共享资源”，父子树解决“谁负责回收谁”，进程组和会话解决“哪些进程作为一个作业被统一管理”。moss 没有把它们压进同一张表，而是分别维护，再通过全局索引提供查询。

2.2.3

Linux 不使用单一 UID 表示进程身份，而是维护 real、effective、saved 三个值。real ID 表示身份来源，effective ID 用于当前权限判断，saved ID 允许进程在受限范围内恢复先前身份。GID 采用相同结构。

moss 将 UID/GID 三元组放入 `ProcessData`，并让 `setreuid`、`setresuid` 等接口真正修改这些状态。非特权进程只能在已有三元组中切换，特权进程可以设置新的值。这样，凭据不再只是系统调用的返回结果，而成为程序执行、信号发送和 ptrace 权限检查共同使用的进程属性。

2.2.4

进程管理不是若干独立系统调用的集合，而是一条连续的生命周期：

> 创建 `clone` → 执行 `exec` → 退出 `exit` → 观察与回收 `wait`

创建阶段建立身份、资源和父子关系；exec 替换程序映像，但保留 PID、父子关系和进程组；exit 释放运行资源并留下退出状态；wait 读取状态并完成最终回收。moss 的多项修改都围绕这条闭环展开，尤其重视失败路径和中间状态，而不仅是系统调用成功时的主路径。

2.3 关键数据结构

2.3.1 Thread ProcessData

`Thread` 和 `ProcessData` 是本模块最重要的两类数据。下面的代码省略了与本节无关的字段，只保留二者的关系。

```rust
pub struct Thread {
    // 共享进程级资源
    pub proc_data: Arc<ProcessData>,
    // 线程私有状态
    pub signal: Arc<ThreadSignalManager>,
    pub exit: Arc<AtomicBool>,
}

pub struct ProcessData {
    // 进程身份及组织关系
    pub proc: Arc<Process>,
    // 线程共享的进程资源
    pub aspace: Arc<Mutex<AddrSpace>>,
    pub scope: RwLock<Scope>,
    pub wait_status: SpinNoIrq<Option<ProcessWaitStatus>>,
    uid: AtomicU32,
    euid: AtomicU32,
    suid: AtomicU32,
}
```

代码 2-1　线程私有状态与进程共享状态

同一进程中的线程持有同一个 `Arc<ProcessData>`，因此能看到相同的地址空间、文件资源和凭据。`Thread` 自身则保存只与当前执行流相关的数据。moss 新增字段时继续遵循这一边界，没有另建一套进程控制块。

2.3.2

StarryOS 原有退出码可以描述进程终止，却不适合表示暂时停止或继续运行。moss 增加 `ProcessWaitStatus`：

```rust
pub enum ProcessWaitStatus {
    // 被信号暂停，但进程仍然存在
    Stopped(Signo),
    // 从停止状态恢复
    Continued(Signo),
}
```

代码 2-2　进程停止与继续状态

退出状态仍由 `Process` 保存；停止和继续状态则独立记录在 `ProcessData` 中。这样，父进程可以区分“子进程已经结束”和“子进程仍然存在，但状态发生变化”，也便于 `waitpid` 与 `waitid` 按不同选项消费事件。

2.3.3

内核需要按 TID、PID、PGID 和 SID 查找对象。moss 使用四张弱引用表分别索引线程、进程、进程组和会话：

索引表	键	作用
TASK_TABLE	TID	查找具体线程和线程私有状态
PROCESS_TABLE	PID	查找进程共享资源和父子关系
PROCESS_GROUP_TABLE	PGID	查找进程组并执行组级操作
SESSION_TABLE	SID	查找会话及其进程组层级

表 2-2　进程线程对象的全局索引

表中保存弱引用，不会因为全局索引而阻止对象释放。moss 还修正了新建进程组和会话后的登记过程，避免对象已经创建，却无法通过 PGID 或 SID 查到。

2.4 进程与线程创建

2.4.1 fork　　clone

`fork` 在 moss 中并未缺席。在 x86_64 平台上，`sys_fork` 会调用 `sys_clone`，其中不设置资源共享标志，只指定子进程退出时发送 `SIGCHLD`。因此，`fork` 创建的是拥有独立 `ProcessData` 的子进程；`clone` 和 `clone3` 则允许用户通过标志决定创建进程还是线程，以及资源如何继承。

这些入口最终都把参数整理为 `CloneArgs`，交给 `CloneArgs::do_clone` 执行。带有 `CLONE_THREAD` 时，新任务复用当前 `ProcessData`；否则调用 `Process::fork` 建立子进程及父子关系，再创建新的 `ProcessData`。地址空间、文件表、文件系统上下文和信号处理动作分别由 `CLONE_VM`、`CLONE_FILES`、`CLONE_FS` 和 `CLONE_SIGHAND` 决定共享或复制。

标志	作用
CLONE_THREAD	加入调用者的线程组，共享同一个 ProcessData
CLONE_VM	共享虚拟地址空间
CLONE_FILES	共享文件描述符表
CLONE_FS	共享文件系统上下文
CLONE_SIGHAND	共享信号处理动作表

表 2-3　进程线程创建中的主要 clone 标志

这些标志可以组合使用，但并非任意组合都合法。`CLONE_THREAD` 决定新任务是否仍属于原进程，其余标志主要控制资源如何继承，或指定创建前后的附加操作。

2.4.2

moss 补充了 clone 标志之间的依赖关系。例如，线程必须共享地址空间和信号处理动作，共享信号处理动作又必须共享地址空间：

```rust
// 线程必须共享地址空间和信号处理动作
if flags.contains(CloneFlags::THREAD)
    && !flags.contains(CloneFlags::VM | CloneFlags::SIGHAND)
{
    return Err(AxError::InvalidInput);
}
// 共享信号处理动作必须共享地址空间
if flags.contains(CloneFlags::SIGHAND)
    && !flags.contains(CloneFlags::VM)
{
    return Err(AxError::InvalidInput);
}
```

代码 2-3　clone 标志组合校验

对于 clone3，moss 进一步检查结构体最小长度、栈地址与栈大小的配对关系，以及超出当前结构体版本的尾部字节。额外字节必须可读且全部为零，使旧内核能够安全接收新版本用户态传入的结构。

2.4.3

创建子进程需要建立进程对象、线程记录、父子关系和 pidfd，最后才把新任务加入系统。若 pidfd 分配或写回用户地址失败，前面建立的状态不能继续保留。

moss 在任务真正启动前设置回滚路径。发生错误时，内核移除刚加入的线程记录，并在其成为最后一个线程时释放新进程。这样，系统调用返回失败后不会留下“有 PID、无可运行线程”的半初始化对象。

但是当前 CLONE_VFORK 尚未完整暂停父进程；namespace 标志也只完成识别和元数据继承。二者在接口上可被处理，但不能视为完整实现。

2.5　程序执行

2.5.1　execve

StarryOS 的 execve 已经能够装载 ELF 文件，替换地址空间和用户上下文，更新程序路径与命令行，重置信号处理动作，并关闭带有 CLOEXEC 标志的文件描述符。exec 不创建新进程，因此 PID、父子关系、进程组和会话保持不变。

基线的不足主要在入口形式和执行约束。它只能按路径执行文件，对文件是否允许执行、是否位于 MS_NOEXEC 挂载点以及是否正在被写入缺少完整判断。

2.5.2　execveat

execveat 用于在指定目录文件描述符下执行程序。路径为相对路径时，内核以 dirfd 指向的目录为起点；指定 AT_EMPTY_PATH 时，还可以直接执行文件描述符所指向的文件。AT_SYMLINK_NOFOLLOW 则要求目标为符号链接时拒绝执行。

传统 execve 只能接收路径，绝对路径依赖进程看到的文件系统，相对路径又依赖当前工作目录。程序若已经打开了可信目录或目标文件，仍需重新拼接并解析路径，期间目录项可能被改名或替换。execveat 可以从稳定的目录文件描述符开始解析，或使用 AT_EMPTY_PATH 执行已经打开的文件，减少重复路径解析带来的竞态，也便于容器运行时、动态加载器等程序在受控目录中启动目标。因此，完整的 Linux 执行接口不能只停留在 execve。

moss 增加该入口后，将 execve 与 execveat 汇入同一条 execve_common 路径，后续装载和状态更新仍由一套逻辑完成。

在替换旧地址空间之前，moss 先完成执行条件检查：

```
// 挂载点禁止执行
if exec_loc.mount_flags() & MS_NOEXEC != 0 {
    return Err(AxError::PermissionDenied);
}
// 按有效 UID/GID 检查执行位
if !has_exec_permission(exec_loc, proc_data.euid(), proc_data.egid())? {
    return Err(AxError::PermissionDenied);
}
// 拒绝正在被写入的可执行文件
if is_text_busy(exec_loc) {
    return Err(AxError::from(LinuxError::ETXTBSY));
}
```

代码 2-4　程序执行前的约束检查

这些检查全部通过后，内核才装载新程序并提交地址空间。若解析、权限或装载失败，原程序仍可继续运行。成功后，内核还会清除旧程序遗留的线程退出指针，避免新地址空间继续引用失效地址。

2.5.3　　exec

Linux 允许多线程进程中的一个线程调用 exec，并在成功后终止其他线程，只保留调用者。moss 当前尚未实现这一收敛过程，多线程进程调用 exec 时会返回失败。这是程序执行路径中仍然明确存在的限制。

2.6　退出与资源回收

2.6.1

一个进程可以包含多个线程，因此某个线程退出并不等于整个进程结束。内核先清理该线程自己的 robust futex 和 `clear_child_tid`，再把它从进程的线程集合中移除。如果还有其他线程存活，共享的地址空间、文件表和 `ProcessData` 都继续保留。

只有最后一个线程退出时，内核才结束整个进程。此时进程进入僵尸状态，保存退出码，向父进程发送退出信号并唤醒正在等待的父进程。moss 还在这一阶段释放文件锁、租约、共享内存记录和 ptrace 关系。进程对象不会立即删除，要等父进程调用 `wait` 读取退出状态后才完成回收。

2.6.2　exit_group

2.6.1 说明了最后一个线程退出的时候如何结束进程，但 `exit_group` 需要主动让进程中的全部线程退出。moss 先标记线程组正在整体退出，再向其他存活线程发送终止信号。各线程仍按 2.6.1 的路径完成清理，最后离开的线程负责执行进程级退出工作。

普通 `exit` 只结束调用线程，`exit_group` 则结束整个进程。这一区别对多线程程序尤其重要。

2.6.3

进程结束后，还要处理它尚未退出的子进程。子进程不能随父进程一起删除，否则它们之后退出时将无人负责回收。StarryOS 会把这些子进程从原父进程的 `children` 表中移出，改由 init 进程托管：

原父进程 → 子进程　　变为　　init → 子进程

子进程本身仍继续运行，原有 PID、进程组和会话不变；变化的只是父进程。它们以后成为僵尸进程时，由 init 调用 `wait` 完成回收。

2.7 子进程状态与等待

2.7.1

StarryOS 的 `waitpid` 主要观察僵尸子进程。moss 在此基础上增加停止和继续事件，使父进程能够区分三类情况：子进程退出、子进程停止、子进程继续运行。

`WUNTRACED`、`WCONTINUED` 和 `WNOHANG` 是 `waitpid`、`wait4` 的 `options` 参数标志：

标志	语义
`WUNTRACED`	子进程因作业控制信号停止时，允许立即报告该状态
`WCONTINUED`	子进程收到 `SIGCONT` 恢复运行时，允许报告继续状态
`WNOHANG`	当前没有可报告状态时立即返回 0，不阻塞调用进程

表 2-4　waitpid 与 wait4 的主要 options 标志

`waitid` 使用 `WSTOPPED` 表示报告停止状态，并配合 `WEXITED`、`WCONTINUED` 和 `WNOWAIT` 使用。moss 内部将 `WSTOPPED` 与同值的 `WUNTRACED` 统一处理。未被当前选项选中的状态会继续保留，等待后续调用读取。

2.7.2 waitpid

`waitpid` 的 pid 参数不仅可以表示一个具体子进程，也可以选择任意子进程或某个进程组。moss 将参数统一转换为 `Any`、`Pid` 和 `Pgid` 三种内部形式，再对直接子进程进行筛选。找不到符合条件的子进程时返回 `ECHILD`，而不是进入无意义的等待。

```
let target = match pid {
    -1 => WaitPid::Any, // 任意直接子进程
    0 => WaitPid::Pgid(proc.group().pgid()),
    1.. => WaitPid::Pid(pid as _),
    _ => WaitPid::Pgid(pid.checked_neg()? as _),
};

// waitpid 只能等待调用进程的直接子进程
let children = proc.children().into_iter()
    .filter(|child| target.apply(child))
    .collect::<Vec<_>>();
if children.is_empty() {
    return Err(LinuxError::ECHILD.into());
}
```

代码 2-5　waitpid 的目标转换与筛选

2.7.3 waitid　pidfd

moss 增加 `waitid`，支持 `P_ALL`、`P_PID`、`P_PGID` 和 `P_PIDFD`。不同目标最终都转换为一组待观察的子进程：

```rust
let (children, pidfd_nonblocking) = match idtype as u32 {
    // 全部直接子进程
    P_ALL => (proc.children(), false),
    // 按 PID 筛选
    P_PID => (proc.children().into_iter()
        .filter(|child| child.pid() == id).collect(), false),
    P_PIDFD => {
        // 由 pidfd 取得稳定进程引用
        let pidfd = PidFd::from_fd(id as _)?;
        let child_pid = pidfd.process_data()?.proc.pid();
        (proc.children().into_iter()
            .filter(|child| child.pid() == child_pid).collect(),
         pidfd.nonblocking())
    }
    _ => return Err(AxError::InvalidInput),
};
```

代码 2-6　waitid 的目标选择逻辑

实际代码在各分支中完成筛选，并根据退出、信号终止、停止或继续填写 `CLD_EXITED`、`CLD_KILLED`、`CLD_STOPPED` 等 `siginfo_t` 类型。`WNOWAIT` 允许父进程只查看状态而不回收目标。

pidfd 本身来自 StarryOS。moss 的增量是把它接入 `waitid(P_PIDFD)`，并补充 `/proc` fdinfo。相比直接保存 PID，pidfd 是一个稳定的文件描述符引用，不会因为 PID 数值被复用而指向另一个进程。

2.8　进程组与会话

2.8.1　PID　PGID　SID

进程组和会话用于组织多个相关进程。每个进程有 PID，所属进程组有 PGID，进程组所属会话有 SID。通常，组长的 PID 等于 PGID，会话首进程的 PID 等于 SID。

这种层级常见于 Shell：一条管道中的多个进程属于同一进程组，一个终端登录环境中的多个进程组属于同一会话。它们仍是独立进程，但可以按组查询、等待或发送信号。

在实现上，moss 沿用 StarryOS 的对象关系：`Process` 保存自身 PID 和所属的 `ProcessGroup`，`ProcessGroup` 保存 PGID、所属 `Session` 及组内进程，`Session` 再保存 SID 和会话内的进程组。

```rust
pub struct Process {
    pid: Pid,
    // 每个进程只属于一个进程组
    group: SpinNoIrq<Arc<ProcessGroup>>,
}

pub struct ProcessGroup {
    pgid: Pid,
    // 进程组归属于一个会话
    session: Arc<Session>,
    // 记录组内的所有进程
    processes: SpinNoIrq<WeakMap<Pid, Weak<Process>>>,
}

pub struct Session {
    sid: Pid,
    // 一个会话可以包含多个进程组
    process_groups: SpinNoIrq<WeakMap<Pid, Weak<ProcessGroup>>>,
}
```

代码 2-7　进程、进程组与会话的对象关系

因此，查询一个进程的三个标识时，内核沿对象引用逐层取得结果：

```rust
// 先按 PID 找到进程对象
let proc = get_process_data(pid)?.proc;
let pid  = proc.pid();
// 沿所属关系取得进程组和会话标识
let pgid = proc.group().pgid();
let sid  = proc.group().session().sid();
```

代码 2-8　PID、PGID 与 SID 的逐层查询

反向操作则依靠成员表完成。例如，向进程组发送信号时，内核遍历 `ProcessGroup::processes`，逐个找到组内进程。moss 还通过 `PROCESS_TABLE`、`PROCESS_GROUP_TABLE` 和 `SESSION_TABLE` 提供按 PID、PGID、SID 的全局查询，并补齐新建进程组和会话后的登记。

2.8.2

StarryOS 已提供 `setpgid`、`getpgid` 等基础接口，但新建的进程组没有同步写入全局 PGID 索引。这样，进程虽然已经加入新组，其他系统调用却可能无法通过 PGID 找到该进程组。

moss 根据目标 PGID 分两种情况处理。目标 PGID 等于进程自身 PID 时，内核创建一个新进程组，并立即将它登记到全局索引中。此后，`getpgid`、组信号和按进程组等待等操作都能查到该对象。

目标 PGID 指向已有进程组时，内核先从全局索引中找到目标组，并检查它与当前进程是否属于同一会话。检查通过后，内核将进程从旧组的成员表中删除，加入新组的成员表，同时更新进程自身保存的所属组。若目标组不存在或属于其他会话，则拒绝迁移。

这样修改后，进程保存的所属组、旧组和新组的成员表、全局 PGID 索引始终保持一致。

2.8.3

`setsid` 会同时创建新会话和新进程组。moss 在创建成功后分别登记 `Session` 和 `ProcessGroup`：

```
if let Some((session, group)) = proc.create_session() {
    // setsid 同时创建会话和进程组
    add_session_to_table(&session);
    add_process_group_to_table(&group);
    // SID 等于调用进程的 PID
    Ok(session.sid() as _)
}
```

代码 2-9　新会话与新进程组的创建和登记

当前实现已经能够维护 PID、PGID 和 SID 的基本关系，但终端前台进程组、后台读写限制和完整孤儿进程组规则尚未形成闭环。

2.9　权限与进程控制

2.9.1　UID/GID

Linux 不用一个 UID 同时表示进程的来源和当前权限，而是维护 real、effective、saved 三个值。real UID 表示进程原本属于哪个用户；effective UID 是执行文件访问、程序执行和信号发送等权限检查时使用的身份；saved UID 保存进程曾经拥有的有效身份。GID 也采用相同的三元组。

这样设计是为了支持受控的权限切换。例如，带有 set-user-ID 属性的程序可以先以较高权限完成必要操作，再降低 effective UID；之后若还需执行受保护操作，可以从 saved UID 恢复原来的权限。若只保存一个 UID，降低权限后便无法安全恢复；若允许进程任意设置 UID，又会形成提权漏洞。

StarryOS 基线中的部分身份设置接口只返回成功，没有保存身份变化。moss 在 `ProcessData` 中加入 UID/GID 三元组，并在 `setresuid`、`setresgid` 等接口中检查转换范围。下面代码展示了 UID 路径的最小实现：

```
pub struct ProcessData {
    uid: AtomicU32,  // real UID: 进程的实际身份
    euid: AtomicU32, // effective UID: 当前权限判断使用
    suid: AtomicU32, // saved UID: 保留可恢复的身份
}

let allowed = [old_ruid, old_euid, old_suid];
if old_euid != 0 {
    // 非特权进程不能把三元组改成从未拥有过的身份
    if !valid_id_change(ruid, &allowed)
        || !valid_id_change(euid, &allowed)
        || !valid_id_change(suid, &allowed)
    {
        return Err(eperm());
    }
}

if ruid != UID_GID_NO_CHANGE {
    proc_data.set_real_uid(ruid);
}
if euid != UID_GID_NO_CHANGE {
    proc_data.set_euid(euid);
}
if suid != UID_GID_NO_CHANGE {
    proc_data.set_suid(suid);
}
```

代码 2-10　UID 三元组的保存与受限转换

非特权进程只能在原有三元组中切换，effective UID 为 0 的特权进程才可以设置新的身份。修改后的 effective UID/GID 会参与程序执行权限、`kill` 和 ptrace attach 等检查。凭据因此成为实际权限判断依据，而不再只是查询接口返回的固定数值。

capability 也由固定结果改为保存 effective、permitted、inheritable 和 bounding 集合，并限制进程任意扩大自身能力。

2.9.2 ptrace

moss 新增 ptrace 的基础状态管理，建立 `TRACEME`、`ATTACH`、`DETACH`、停止、继续和 wait 报告之间的关系。attach 时先检查目标是否存在、是否已被跟踪以及调用者是否具有相应权限；目标停止后，跟踪者通过 wait 获取事件。

当前实现只能完成最小控制流程。`PTRACE_SYSCALL` 和 `PTRACE_SINGLESTEP` 暂按普通继续处理，寄存器、目标内存和 siginfo 读写请求返回 `EIO`，因此还不能视为完整的 GDB 后端。

2.9.3 namespace

moss 为 PID、UTS 和 User namespace 保存 inode、父 namespace 和 init PID 等元数据，也能识别 clone 与 unshare 中的相关标志。但当前实现不会建立独立 PID 视图、主机名隔离或 UID 映射，代码也明确将相关路径标记为兼容桩。

因此，这部分只能描述为 namespace 接口和状态预留，不能计作完整的容器隔离能力。

2.10 本章小结

moss 选择沿用 StarryOS 的进程线程框架，围绕四条主线完成加固：以 `Thread` 和 `ProcessData` 组成双层对象模型，以线程归属、父子树和进程组会话维护三类组织关系，以 real、effective、saved ID 构成三元凭据，并把 clone、exec、exit、wait 串成完整生命周期。

在此基础上，moss 补严 `clone3` 的 ABI 和失败回滚，增加 `execveat`，完善退出清理与子进程状态报告，将 pidfd 接入等待接口，并加入有状态的权限和 ptrace 控制。当前还未完成完整 vfork、多线程 exec、namespace 隔离和 ptrace 数据访问，但进程从创建到回收的主要管理关系已经比 StarryOS 基线更完整。